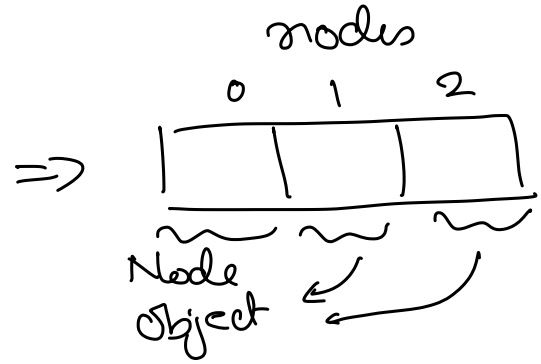


How to use an array to allocate memory for all nodes for a linked list?

Array of objects C++

Node nodes[3];

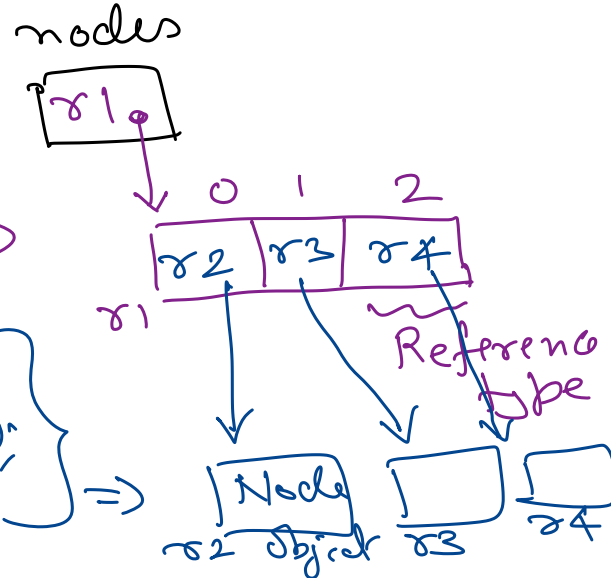


Array of objects JAVA

Node[] nodes;

nodes = new Node[3]; $\Rightarrow$

$\left. \begin{array}{l} \text{for (i = 0; i < 3; ++i)} \\ \text{nodes[i] = new Node();} \end{array} \right\} \Rightarrow$



Object Pool

```
class Node Pool Manager {
```

```
    Node[] nodes;
```

```
    boolean[] isFree;
```

```
    Node createNode();
```

```
    void deleteNode(Node node);
```

```
}
```

AddAtFront(element) - Optimised

- Make space for new elements, say newNode.
- Store element in newNode's data.
- Set newNode's next to head.
- Set head to newNode.
- if tail is empty then
 - Set tail to head.
- Stop.

→ Node newNode =
nodePoolMgr.createNode();

Recursion

Base case



When the solution of a problem is defined as a solution of a subproblem.

In programming - When a function calls itself.

$$n! = \begin{cases} 1, & \text{if } n = 0 \text{ or } 1 \\ n * (n-1)!, & \text{otherwise} \end{cases}$$

Terminating condition

```
int factorial (int n) {  
    if ((n == 0) || (n == 1)) {  
        return 1;  
    }  
    return n * factorial (n-1);  
}
```

Direct vs Indirect recursion.

Infinite recursion and terminating condition/base case.

```
... f1() {  
    :  
    f1(); ← Direct  
    :  
}
```

Recursion

```
... f2() {  
    :  
    f3();  
    :  
}  
... f3() {  
    :  
    f2();  
    :  
}
```

indirect
recursion

```
... f4() {  
    f4(); ← infinite  
    :  
}
```

recursion

⇒ when recursive call
is made before
terminating
condition

Divide and Conquer

If a problem can be divided into smaller problems such that, the solution of smaller problems gives us the solution of bigger problem.

DIVIDE - Divide larger problem into smaller problem.

CONQUER - Solve each smaller problem until they are a base case.

COMBINE (Optional) - Combine solution of smaller problems to find solution of larger problem.

Backtracking

Build solution step by step.

At each step, we discard steps(s) that do not result in a solution.

→ Maze.

→ Knight Tour

→ Sudoku.

→ 8 Queen

Multiply two numbers, without using $*$ operator.

multiply $(x, y) =$

0, if $x=0$ or $y=0$.

x , if $y=1$

y , if $x=1$

$x + \text{multiply}(x, y-1)$,
otherwise.

$$3 * 4 = \underbrace{3 + 3 + 3 + 3}_{4 \text{ times}}$$

$$\underbrace{4 + 4 + 4}_{3 \text{ times}}$$



$$3 + \text{multiply}(3, 3)$$

Assignment: Find quotient of dividing two integers.

Assignment: Find remainder

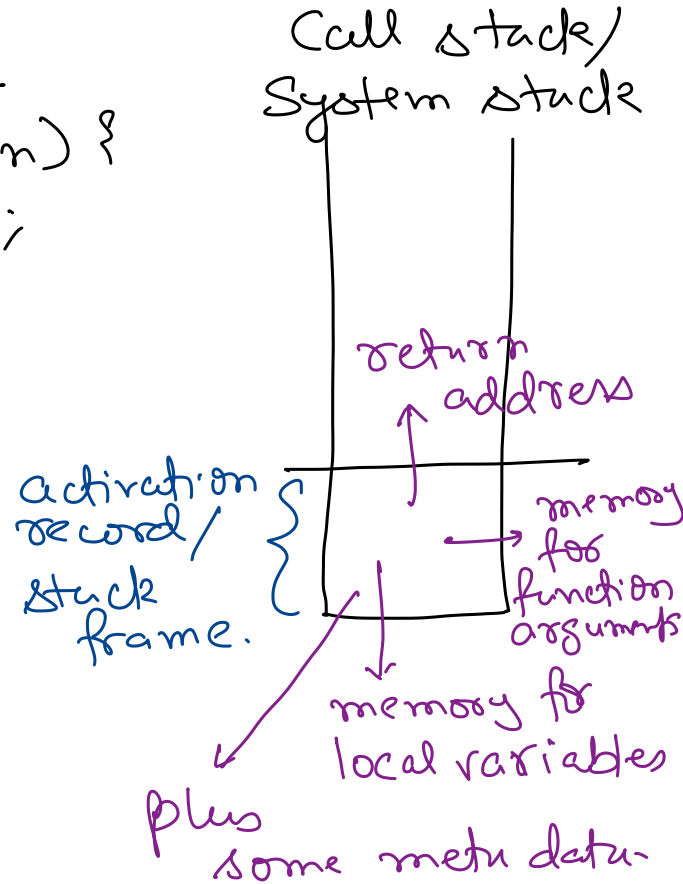
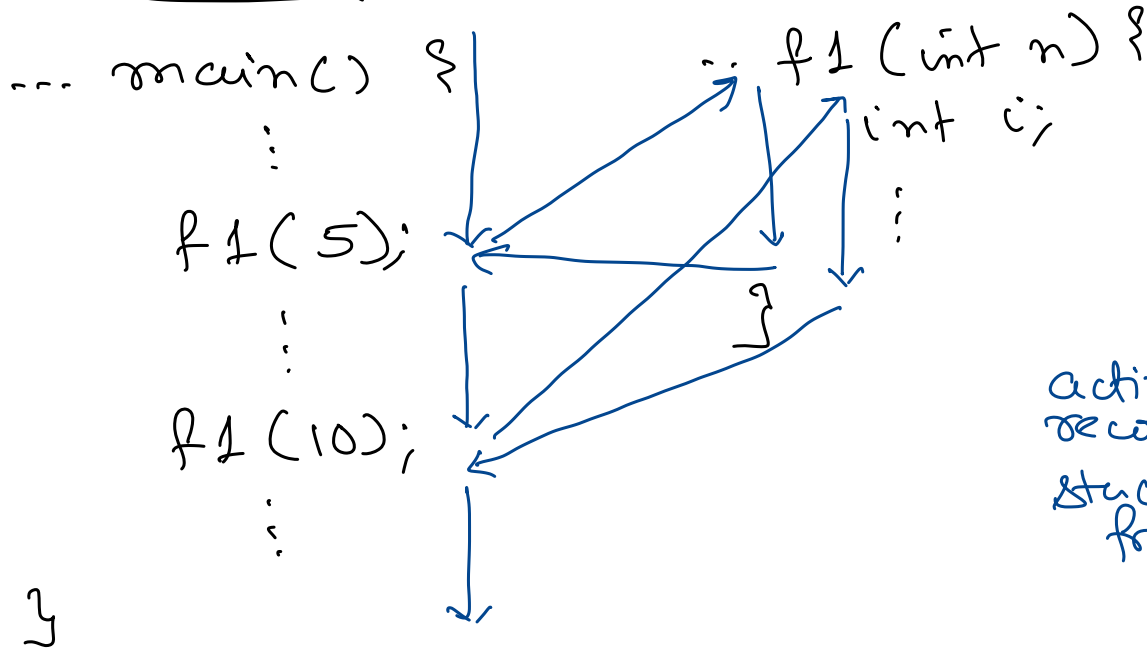
Assignment: Multiply two positive numbers efficiently.

+ve as well as -ve



Assignment: Multiply two integers, efficiently.

How do function call works?



```

int factorial (int n) {
    if ((n == 0) || (n == 1)) {
        return 1;

```

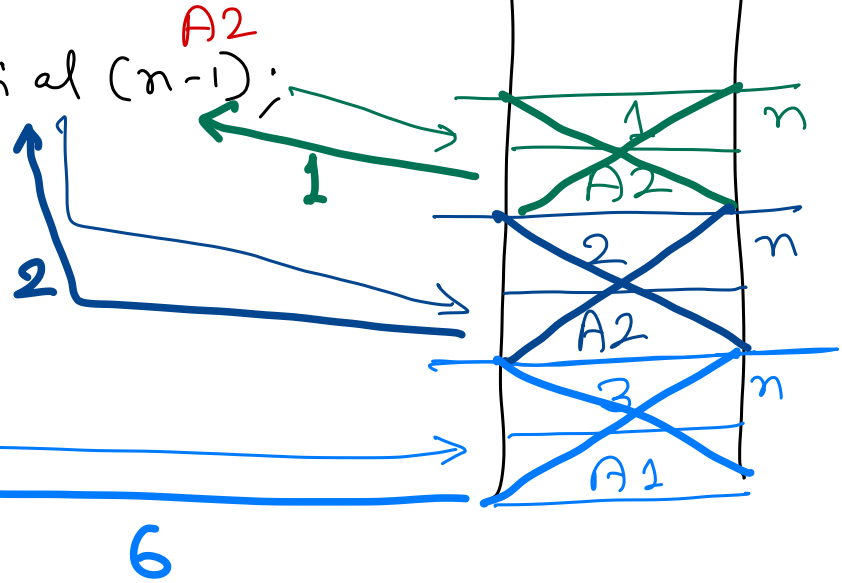
```

        return n * factorial(n-1);
    }

```

}

factorial(3); A1



Stack unwinding

↳ Removing stack frame when function call is over.

Tail Recursion $\rightarrow$ when last statement in function is the recursive call.

... f1() {
 if (...) return ...;
 ...
}

